

Load Balancing

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

How traffic gets spread evenly across many servers – L4 vs L7 balancing, the algorithms that pick a backend, health checks, session persistence, and how to build a load balancer that isn't itself a single point of failure.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 – 2	Why load balancing exists, and L4 vs L7 balancing
3 – 4	Load balancing algorithms, and health checks & failover
5 – 6	Session persistence and SSL/TLS termination
7 – 9	High-availability LB design, a worked example, and reference

How to use this guide: read start to finish for the full picture, or jump to section 3 if you just need the algorithm comparison. Every diagram is numbered and referenced directly in the surrounding text.

Table of Contents

1. What Is Load Balancing & Why It Exists	3
2. L4 vs L7 Load Balancing	4
3. Load Balancing Algorithms	5
4. Health Checks & Failover	6
5. Session Persistence	7
6. SSL/TLS Termination	8
7. High Availability: Avoiding a Single Point of Failure	9
8. Worked Example: Configuring a Load Balancer	10
9. Quick Reference & Glossary	11

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

What Is Load Balancing & Why It Exists

1.1 ONE SERVER ISN'T ENOUGH

A single server can only handle so many simultaneous requests before it runs out of CPU, memory, or network capacity – and if that one server goes down, the service goes down with it. **Load balancing** solves both problems at once: a load balancer sits in front of a pool of identical backend servers, distributing incoming requests across them so no single server is overwhelmed, and automatically routing around any server that fails.

The core trade: load balancing turns "one big, expensive, still-fragile server" into "several smaller, cheaper servers that can scale out as demand grows, and survive individual failures." This pattern – horizontal scaling – is the foundation almost all modern web infrastructure is built on.

1.2 WHAT A LOAD BALANCER ACTUALLY DOES

- **Distributes traffic** – spreads incoming requests across backend servers using a chosen algorithm (Section 3).
- **Monitors health** – continuously checks whether each backend is actually able to serve requests, and stops sending traffic to any that aren't (Section 4).
- **Absorbs failure** – if a backend server crashes or becomes unreachable, traffic simply reroutes to the remaining healthy servers, often without users noticing at all.
- **Enables scaling** – adding capacity becomes as simple as adding another backend server to the pool, rather than replacing hardware with something bigger.

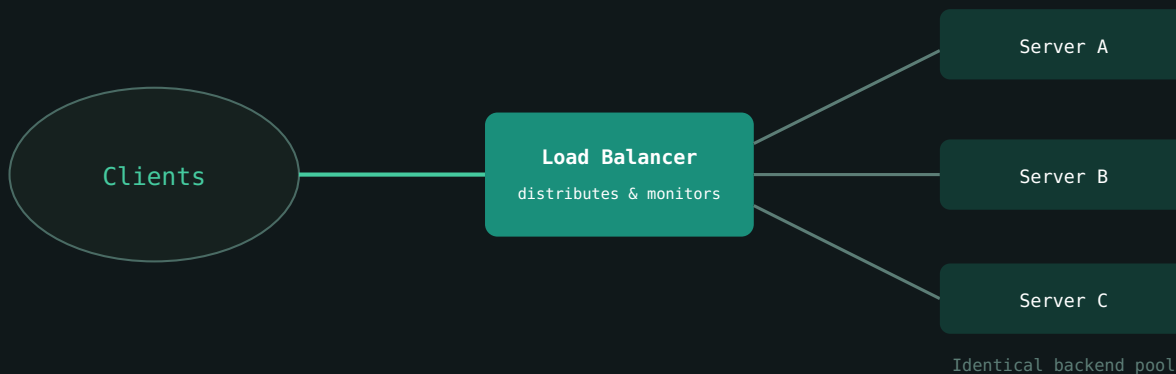


Fig 1.1 – A load balancer sits between clients and an identical pool of backend servers, distributing and monitoring traffic across them.

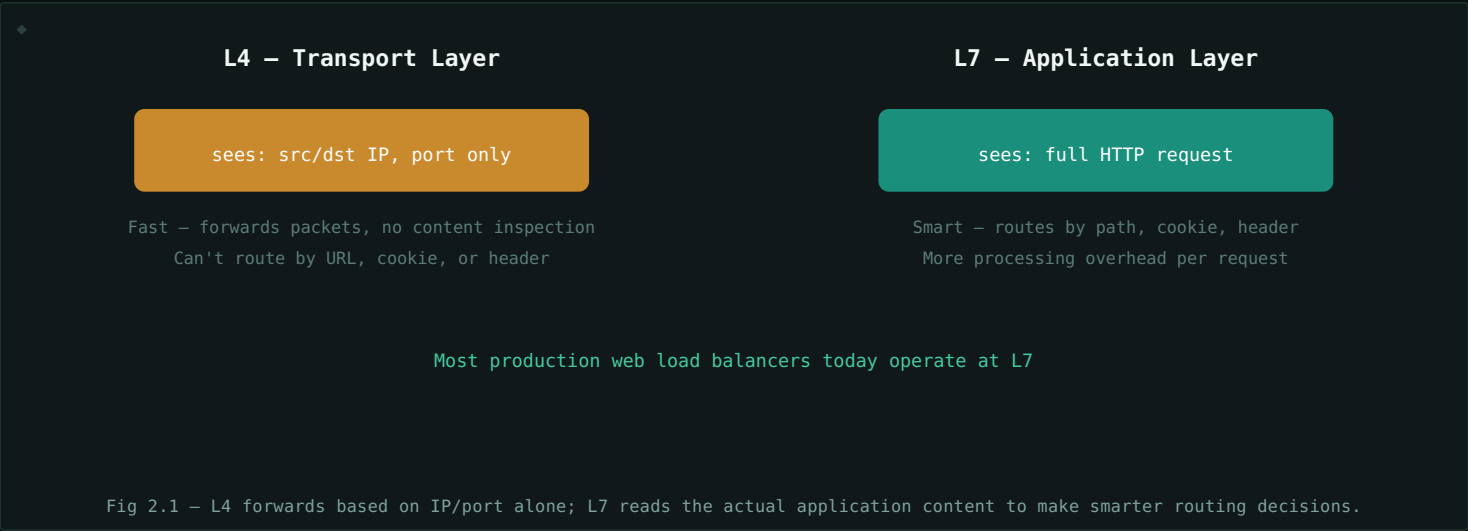
L4 vs L7 Load Balancing

2.1 LAYER 4 (TRANSPORT-LAYER) LOAD BALANCING

An **L4 load balancer** makes its distribution decision based only on Transport-layer information – source/destination IP and port – without looking at the actual content of the traffic at all. It forwards packets (often just rewriting the destination, similar in spirit to the NAT covered in the NAT Deep Dive guide) with very little processing overhead. Fast, protocol-agnostic, but blind to what's actually inside the connection – it can't make decisions based on a URL path, a cookie, or an HTTP header, because it never inspects any of that.

2.2 LAYER 7 (APPLICATION-LAYER) LOAD BALANCING

An **L7 load balancer** terminates the connection and actually reads the application-layer content – HTTP headers, URL paths, cookies – before deciding which backend to forward to. This enables much smarter routing: sending `/api/*` requests to one server pool and `/images/*` to another, or routing based on a session cookie. The cost is more processing overhead per request, since the load balancer has to fully parse the application protocol rather than just glancing at packet headers.



2.3 QUICK COMPARISON

ASPECT	L4	L7
Visibility	IP address and port only	Full HTTP request – path, headers, cookies
Performance	Very fast, minimal overhead	Slower – must parse the application protocol
Routing intelligence	Basic – by IP/port only	Advanced – content-based routing, A/B testing
Typical use	Raw TCP/UDP services, high-throughput needs	Web applications, microservices, APIs

Load Balancing Algorithms

Once traffic reaches the load balancer, it needs a rule for *which specific backend* gets each request. Different algorithms suit different workloads.

3.1 ROUND ROBIN

Requests are distributed to each backend in strict rotating order – server A, then B, then C, then back to A. Simple and predictable, but assumes every backend and every request are roughly equal, which isn't always true if some requests are much heavier than others or servers have different capacities.

3.2 WEIGHTED ROUND ROBIN

Same rotation, but each backend is assigned a weight reflecting its relative capacity – a more powerful server might get a weight of 3 against a smaller server's weight of 1, receiving three requests for every one the smaller server gets. Solves round robin's "treats unequal servers equally" problem.

3.3 LEAST CONNECTIONS

Routes each new request to whichever backend currently has the **fewest active connections**. This adapts naturally to servers that are already busy or slow to finish existing requests, unlike round robin which keeps sending traffic on a fixed schedule regardless of how backed up a server already is.

3.4 LEAST RESPONSE TIME

Combines active connection count with actual measured response latency – routing to the backend that's both lightly loaded *and* currently responding quickly. More adaptive than least connections alone, at the cost of needing continuous latency measurement.

3.5 IP HASH

Computes a hash of the client's source IP address to consistently route the same client to the same backend every time. This is a simple way to achieve session persistence (Section 5) without needing cookies, though it can distribute load unevenly if many clients happen to share an IP (common behind NAT/CGNAT – see the NAT Deep Dive guide, Section 6).

ALGORITHM	ADAPTS TO SERVER LOAD?	BEST FOR
Round Robin	No	Homogeneous backends, uniform request cost
Weighted Round Robin	No (static weights)	Backends with known, differing capacities
Least Connections	Yes – by connection count	Variable request duration/cost
Least Response Time	Yes – by load and latency	Latency-sensitive applications
IP Hash	No	Simple session persistence without cookies

Health Checks & Failover

4.1 WHY HEALTH CHECKS MATTER

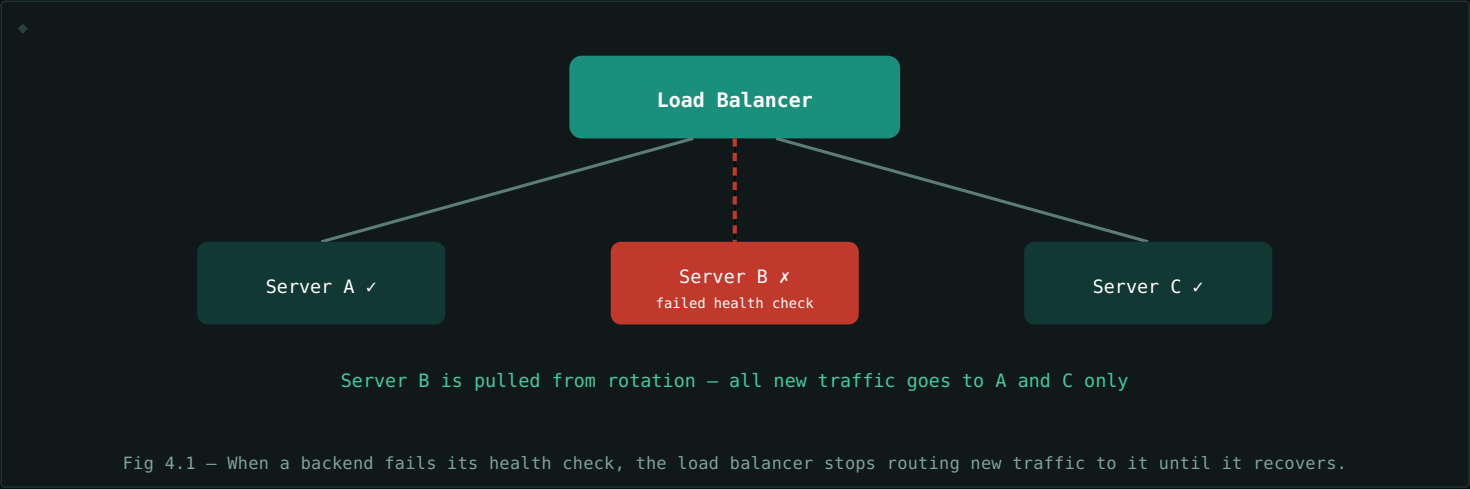
An algorithm (Section 3) is only useful if it's choosing among *healthy* backends. Without health checking, a load balancer would keep sending traffic to a crashed or overloaded server indefinitely, since nothing tells it otherwise. **Health checks** are periodic probes the load balancer sends to each backend to verify it's actually able to serve requests – any backend that fails enough consecutive checks is automatically pulled out of rotation.

4.2 TYPES OF HEALTH CHECKS

TYPE	WHAT IT CHECKS	DEPTH
TCP Check	Can a connection even be established on the port	Shallow – server is "up" but maybe not actually working
HTTP Check	Does a specific URL return an expected status code	Deeper – confirms the application is actually responding
Custom / Deep Check	A dedicated health endpoint verifying database connectivity, dependencies, etc.	Deepest – confirms the whole request path actually works

Shallow checks lie. A TCP check can pass even when the application behind it is completely broken – the port is open, but the app itself might be deadlocked, out of memory, or unable to reach its database. This is why most production systems use an HTTP check against a dedicated `/health` endpoint that the application itself controls, rather than trusting that an open port means the service actually works.

4.3 FAILOVER BEHAVIOR



4.4 AUTOMATIC RECOVERY

Once a failed backend starts passing health checks again – after a crash, deployment, or transient network issue resolves – the load balancer automatically returns it to rotation. This entire cycle typically happens within seconds, often without any human intervention or even user-visible disruption, which is exactly the resilience benefit load balancing was built to provide in the first place.

Session Persistence

5.1 THE PROBLEM: STATEFUL APPLICATIONS

Most load balancing algorithms (Section 3) freely send a client's requests to *any* healthy backend. That's fine if every backend is stateless and interchangeable – but many applications store session data (like a shopping cart, or login state) locally on whichever server first handled that user. If the next request lands on a different backend that never saw that session, the user appears logged out or loses their cart. **Session persistence** (also called "sticky sessions") solves this by consistently routing a given client's requests to the same backend for the duration of their session.

5.2 COMMON PERSISTENCE METHODS

METHOD	HOW IT WORKS	TRADE-OFF
Cookie-Based	Load balancer inserts a cookie identifying which backend handled the first request; all future requests with that cookie go to the same backend	Most reliable, requires L7 (Section 2)
IP Hash	Client's source IP consistently hashes to the same backend	Simple, but breaks down behind shared/CGNAT IPs
SSL Session ID	Reuses the TLS session identifier to pin a client to a backend	Works below the application layer, ties persistence to the TLS session lifetime

5.3 THE BETTER LONG-TERM FIX: EXTERNALIZE SESSION STATE

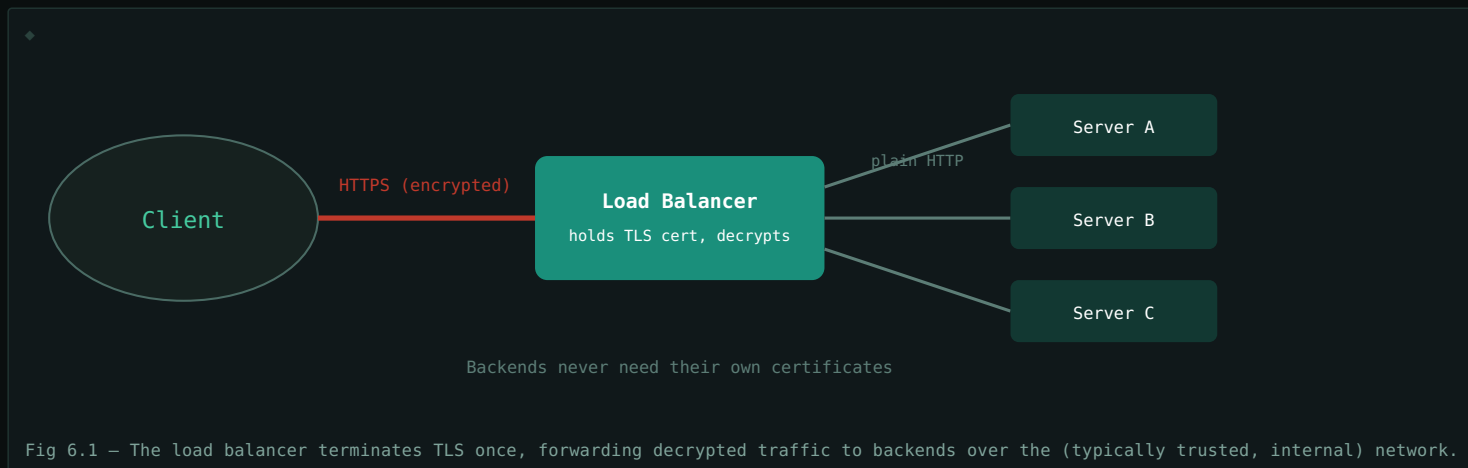
The most robust approach isn't really a load-balancing technique at all – it's an application design choice: store session state in a shared, external location (like Redis or a database) that every backend can read, rather than on any individual server. This makes every backend truly stateless and interchangeable again, letting the load balancer freely use whichever algorithm best distributes load (Section 3) without needing to solve session affinity at the load-balancing layer at all. Sticky sessions are a practical workaround; externalized session state is the underlying fix.

Sticky sessions have a cost: pinning a client to one backend undermines the even distribution the load balancer is otherwise trying to achieve – if that backend gets a disproportionate share of long-lived, "sticky" sessions, it can end up more loaded than the others despite the load balancer's best efforts.

SSL/TLS Termination

6.1 WHAT TERMINATION MEANS

SSL/TLS termination is when the load balancer itself decrypts incoming HTTPS traffic, then forwards the request to backend servers as plain, unencrypted HTTP over the internal network. The load balancer holds the TLS certificate and private key, handling the computationally expensive encryption/decryption work once, centrally – rather than requiring every single backend server to manage its own certificate and perform its own TLS handshake.



6.2 WHY TERMINATE AT THE LOAD BALANCER

- **Centralized certificate management** – one place to install, renew, and rotate TLS certificates, instead of every backend server.
- **Reduced backend CPU load** – TLS handshakes and encryption are computationally expensive; offloading this frees up backend capacity for actual application work.
- **Enables L7 routing** – the load balancer needs to read the decrypted HTTP content anyway to make content-based routing decisions (Section 2), which requires termination.

6.3 SSL PASSTHROUGH: THE ALTERNATIVE

Some designs instead use **SSL passthrough**, where the load balancer forwards encrypted traffic untouched all the way to the backend, which performs its own decryption. This keeps traffic encrypted end-to-end (sometimes a compliance requirement) but sacrifices L7 routing capability, since the load balancer can no longer see the request's actual content, and pushes the TLS processing cost back onto every backend server individually.

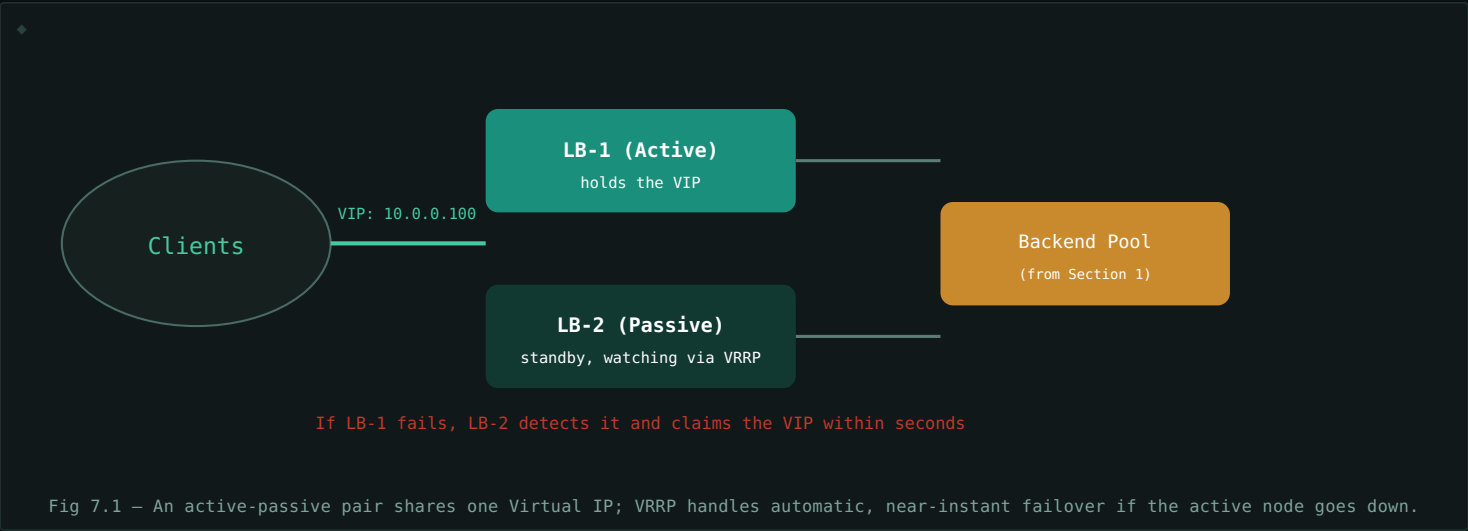
High Availability: Avoiding a Single Point of Failure

7.1 THE IRONY OF LOAD BALANCERS

A load balancer exists to eliminate a single point of failure among backend servers – but if there's only *one* load balancer, it has simply become the new single point of failure. Every request now depends on that one device; if it goes down, every backend behind it becomes unreachable regardless of how healthy they are. Production load-balancing designs need to solve this problem too.

7.2 ACTIVE-PASSIVE WITH VRRP/KEEPALIVED

Two load balancers share a **Virtual IP (VIP)** – the address clients actually connect to. One is **active** and holds the VIP, handling all traffic; the other is **passive**, continuously monitoring the active one's health via a protocol like **VRRP (Virtual Router Redundancy Protocol)**. If the active load balancer fails, the passive one detects this within seconds and takes over the VIP, becoming active itself – a process called **failover**. Clients experience at most a brief interruption, without needing to know the VIP ever moved to different physical hardware.



7.3 ACTIVE-ACTIVE DESIGNS

Rather than leaving a second load balancer idle as a pure standby, an **active-active** design runs multiple load balancers simultaneously handling live traffic – often using DNS round robin or an upstream router (ECMP – Equal-Cost Multi-Path routing) to spread client connections across all of them. This makes better use of the standby hardware's capacity, at the cost of more complex state synchronization if session persistence (Section 5) needs to survive a failover between load balancer nodes.

DESIGN	STANDBY UTILIZATION	COMPLEXITY
Active-Passive	Idle until failover	Simpler – one clear active node at a time
Active-Active	Fully utilized	Higher – needs traffic distribution and state sync across nodes

Worked Example: Configuring a Load Balancer

A simplified, NGINX-style configuration implementing an L7 load balancer (Section 2) with least-connections balancing (Section 3), health checks (Section 4), and TLS termination (Section 6) – pulling together most of what this guide has covered.

8.1 DEFINE THE BACKEND POOL

```
$ upstream backend_pool {  
$   least_conn;  
$   server 10.0.0.11:8080 weight=1;  
$   server 10.0.0.12:8080 weight=1;  
$   server 10.0.0.13:8080 weight=2;  
$ }
```

Three backends, least-connections algorithm, with Server 13 weighted to receive roughly twice the traffic of the other two – perhaps because it runs on more capable hardware.

8.2 CONFIGURE TLS TERMINATION & ROUTING

```
$ server {  
$   listen 443 ssl;  
$   ssl_certificate /etc/ssl/cert.pem;  
$   ssl_certificate_key /etc/ssl/key.pem;  
$   location / {  
$     proxy_pass http://backend_pool;  
$   }  
$ }
```

The load balancer listens on 443, terminates TLS using the configured certificate (Section 6), and proxies decrypted requests to the backend pool defined above.

8.3 ADD ACTIVE HEALTH CHECKS

```
$ health_check interval=5s  
$     fails=3  
$     passes=2  
$     uri=/health;
```

Probes each backend's `/health` endpoint every 5 seconds; a backend is marked unhealthy and pulled from rotation after 3 consecutive failures (Section 4), and must pass 2 consecutive checks before being trusted with traffic again.

In practice: exact syntax varies significantly between platforms (NGINX, HAProxy, cloud-native load balancers like AWS ALB), but the underlying structure – define a backend pool with an algorithm, configure how traffic reaches it, and layer health checking on top – is consistent across virtually all of them.

Quick Reference & Glossary

TERM	DEFINITION
Load Balancer	A device or service that distributes incoming traffic across a pool of backend servers.
Backend / Upstream	One of the servers in the pool a load balancer distributes traffic to.
L4 Load Balancing	Distribution based on Transport-layer info (IP, port) only, without inspecting content.
L7 Load Balancing	Distribution based on Application-layer content – URL, headers, cookies.
Round Robin	An algorithm rotating requests evenly across backends in fixed order.
Least Connections	An algorithm routing each request to the backend with the fewest active connections.
Health Check	A periodic probe verifying a backend is able to serve requests.
Failover	Automatically rerouting traffic away from a failed component to a healthy one.
Session Persistence	Consistently routing a client's requests to the same backend ("sticky sessions").
SSL/TLS Termination	Decrypting HTTPS traffic at the load balancer before forwarding to backends.
SSL Passthrough	Forwarding encrypted traffic untouched, letting the backend handle decryption itself.
VIP	Virtual IP – a shared address clients connect to, that can move between redundant load balancers.
VRRP	Virtual Router Redundancy Protocol – lets a standby node detect failure and take over a VIP.
Active-Passive	An HA design where one node handles traffic while a standby waits to take over on failure.
Active-Active	An HA design where multiple nodes handle live traffic simultaneously.

End of guide. Load balancing is really two problems solved by one component: spreading load evenly (Section 3) and surviving individual failures (Section 4) – and the load balancer itself needs the same failure-survival treatment (Section 7) that it provides to everything behind it.